VOLUME 8 OF 12

IT STUDY HUB

COMPLETE JAVA PROGRAMMING SERIES

MODULE 8

MODERN JAVA & FUNCTIONAL PROGRAMMING

Authored by:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

Welcome to Module 8

1. About This Module

Welcome to Volume 8 of the IT Study Hub Complete Java Programming Series. In Module 7, we learned how to store vast amounts of data using the Collections Framework. However, processing that data using traditional `for` loops and `if` statements requires writing huge blocks of repetitive "boilerplate" code.

This module brings your skills into the modern era. Starting with Java 8, Java introduced **Functional Programming** concepts. You will learn to write code that focuses on *what* to achieve rather than *how* to step through it manually. This shift will make your code drastically shorter, infinitely more readable, and significantly more powerful.

2. Learning Outcomes

- Understand the paradigm of Functional Programming in Java.
- Write concise, anonymous functions using **Lambda Expressions**.
- Master the four core **Functional Interfaces**: `Predicate`, `Consumer`, `Supplier`, and `Function`.
- Utilize **Method References** as a shorthand for Lambdas.
- Process massive collections effortlessly using the **Stream API**.
- Distinguish between Intermediate (`filter`, `map`, `sorted`) and Terminal (`collect`, `reduce`, `forEach`) stream operations.
- Eliminate `NullPointerExceptions` using the **Optional** class.
- Handle dates and times securely using the modern **Date and Time API**.
- Model immutable data effortlessly using Java 14+ **Records** and strict hierarchies with **Sealed Classes**.

3. Module Coverage

We begin by dismantling traditional methods into sleek Lambda expressions. We will then feed these lambdas into Streams to create elegant data-processing pipelines. We will replace the outdated `java.util.Date` with the robust `java.time` package. We will explore cutting-edge features like Records and Pattern Matching. Finally, you will synthesize everything in a "Java Data-Processing Application" project, followed by our extensive assessment section.

IT STUDY HUB

4. Main Notes: Lambdas and Functional Interfaces

Prior to Java 8, Java was purely Object-Oriented. Everything had to be wrapped inside a Class, even a simple math function. Functional programming allows us to treat functions as "first-class citizens"—you can pass a function directly into a variable or another method without building an entire class for it.

IT STUDY HUB

Concept: Lambda Expressions

A short block of code which takes parameters and returns a value. It is an anonymous function (a method without a name or class).

> ***Real-Life Analogy:***
> *Normally, if you want a sandwich, you have to build a whole restaurant (Class), hire a chef (Method), and order the sandwich. A Lambda is a vending machine—you just insert the ingredients and it instantly hands you the sandwich without building a restaurant.*

> **Simple Explanation:**
> A Lambda expression allows you to write methods in an extremely condensed syntax. It uses the arrow operator `->` which separates the input parameters from the code body.

Syntax / Structure:

```
(parameters) -> { body }
// or if only one line:
(parameter) -> expression
```

Example Code:

```
import java.util.ArrayList;
import java.util.List;

public class LambdaDemo {
    public static void main(String[] args) {
        List names = new ArrayList<>();
        names.add("Alice");
        names.add("Bob");
        names.add("Charlie");

        // TRADITIONAL WAY (Enhanced For-Loop)
        System.out.println("Traditional:");
        for(String name : names) {
            System.out.println(name);
        }

        // MODERN WAY (Lambda Expression)
```

```
        System.out.println("\nLambda:");
        names.forEach(name -> System.out.println(name));
    }
}
```

```
Expected Output:
Traditional:
Alice
Bob
Charlie

Lambda:
Alice
Bob
Charlie
```

Code Explanation:
The `.forEach()` method accepts a Lambda. We defined a temporary variable `name`, used the arrow `->`, and told Java what to do with it: `System.out.println(name)`. We did in one line what previously took three.

> ***Real-world Use:***
> *Passing custom behavior into UI buttons (e.g., `button.onClick(event -> submitForm())`).*

> **Common Mistakes:**
> • Forgetting that if you have multiple lines of code in your lambda, you MUST use curly braces `{}` and explicitly use the `return` keyword (if returning data).

> **Best Practices:**
> • Keep lambdas short. If your lambda is more than 3-4 lines, it should probably be moved to a regular named method for readability.

Practice Task:
Create an ArrayList of Integers (1, 2, 3). Use `.forEach()` and a lambda to print each number multiplied by 10.

Lambda Expression Structure

```
(x, y) → x + y
```

1. Parameters
The inputs (just like a normal method). Data types are usually inferred automatically.

2. Arrow Token
Separates the inputs from the execution logic.

3. Body
The code to execute. Can be a single expression or a full {} block.

IT STUDY H

Concept: Functional Interfaces

An interface that contains EXACTLY ONE abstract method. They act as the 'data type' for Lambda expressions.

> ***Real-Life Analogy:***
> *A functional interface is a highly specialized electrical socket that accepts exactly one specific type of plug (the Lambda).*

> **Simple Explanation:**
> For a Lambda to work, Java needs to know what 'type' of function it is. Java provides built-in Functional Interfaces in the `java.util.function` package. The `@FunctionalInterface` annotation is used to guarantee the interface only has one abstract method.

Syntax / Structure:

```
@FunctionalInterface
interface MathOperation {
    int operate(int a, int b);
}
```

Example Code:

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}

public class FunctionalInterfaceDemo {
    public static void main(String[] args) {

        // Lambda acting as the 'calculate' method body
        Calculator addition = (a, b) -> a + b;
        Calculator multiplication = (a, b) -> a * b;

        System.out.println("10 + 5 = " + addition.calculate(10, 5));
        System.out.println("10 * 5 = " + multiplication.calculate(10, 5));
```

```
    }
}
```

```
Expected Output:
10 + 5 = 15
10 * 5 = 50
```

Code Explanation:
We defined a contract `Calculator` that requires an `int` return and two `int` parameters. The Lambda `(a, b) -> a + b` perfectly matches this contract, so Java allows us to assign the Lambda directly to the `Calculator` variable.

> ***Real-world Use:***
> *Instead of creating a whole `AdditionClass` and `MultiplicationClass`, we use one interface and define the specific behavior on the fly.*

Common Mistakes:

- Trying to use a lambda with an interface that has TWO or more abstract methods. The compiler will reject it because it doesn't know which method the lambda is supposed to implement.

Best Practices:

- Use the `@FunctionalInterface` annotation when creating your own. It forces the compiler to warn you if someone accidentally adds a second method to the interface later.

---

5. The Big Four Built-in Functional Interfaces

You rarely need to write your own functional interfaces because Java 8 provides four generic ones that cover 99% of use cases. They are heavily used in Streams.

Interface	Input	Output	Purpose (Analogy)
Predicate<T>	1 Object (T)	`boolean`	The **Filter**. Evaluates a condition. (e.g., Is this apple rotten? True/False).
Consumer<T>	1 Object (T)	`void`	The **Eater**. Takes an item, does something with it, returns nothing. (e.g., Print this text).
Supplier<T>	None	1 Object (T)	The **Factory**. Takes nothing, produces an object. (e.g., Generate a random ID).
Function<T, R>	1 Object (T)	1 Object (R)	The **Transformer**. Takes Type T, changes it, returns Type R. (e.g., Takes an Employee object, returns just their Name string).

IT STUDY HUB

Concept: Predicate & Consumer Example

Predicate evaluates to true/false. Consumer executes an action without returning data.

> ***Real-Life Analogy:***
> *A security checkpoint. The Predicate is the metal detector (Does it beep? Yes/No). The Consumer is the security guard who physically searches the bag (takes the bag, does an action, returns nothing).*

Syntax / Structure:

```
Predicate p = (x) -> condition;
Consumer c = (x) -> System.out.println(x);
```

Example Code:

```
import java.util.function.Predicate;
import java.util.function.Consumer;

public class BuiltInInterfaces {
    public static void main(String[] args) {

        // PREDICATE: Takes an Integer, returns true if even
        Predicate isEven = (num) -> num % 2 == 0;

        System.out.println("Is 10 even? " + isEven.test(10));
        System.out.println("Is 7 even? " + isEven.test(7));

        // CONSUMER: Takes a String, prints it uppercase, returns nothing
        Consumer shout = (text) -> System.out.println(text.toUpperCase() + "!");

        shout.accept("hello");
    }
}
```

Expected Output:

```
Is 10 even? true
Is 7 even? false
HELLO!
```

Code Explanation:
We implement `Predicate` using a lambda that checks `% 2`. We trigger it using the built-in `.test()` method. We implement `Consumer` using a lambda that prints text. We trigger it using the built-in `.accept()` method.

Real-world Use:
Predicates are used to `.filter()` lists. Consumers are used in `.forEach()` loops.

Common Mistakes:

- Trying to return a value from a Consumer.
- Forgetting to call `.test()` or `.accept()` to actually execute the lambda.

Best Practices:

- Store complex predicates in variables so they can be reused across different streams.

Practice Task:
Create a `Supplier` that returns a random number using `Math.random()`. Trigger it using `.get()`.

---

6. Method References

IT STUDY HUB

Concept: Method References (::)

A shorthand syntax for a lambda expression that contains ONLY one method call.

> ***Real-Life Analogy:***
> *Instead of saying 'Hey Bob, take this box and put it in the truck' (Lambda), you just point at the truck (Method Reference) and Bob knows exactly what to do.*

> **Simple Explanation:**
> If your lambda expression does nothing but pass its parameter directly into an existing method, you can use the double colon `::` syntax to make the code even shorter.

Syntax / Structure:

```
// Lambda
(x) -> System.out.println(x)

// Method Reference Equivalent
System.out::println
```

Example Code:

```
import java.util.Arrays;
import java.util.List;

public class MethodRefDemo {
    public static void main(String[] args) {
        List cities = Arrays.asList("London", "Tokyo", "New York");

        // Approach 1: Lambda
        System.out.println("Using Lambda:");
        cities.forEach(city -> System.out.println(city));

        // Approach 2: Method Reference
        System.out.println("\nUsing Method Reference:");
        cities.forEach(System.out::println);
    }
}
```

```
Expected Output:
Using Lambda:
London
Tokyo
New York

Using Method Reference:
London
Tokyo
New York
```

Code Explanation:
Because the lambda `city -> System.out.println(city)` does literally nothing but hand the `city` variable to the `println` method, Java allows us to collapse it entirely using `ClassName::MethodName`.

> ***Real-world Use:***
> *Used extensively in Stream processing to keep code incredibly clean and declarative.*

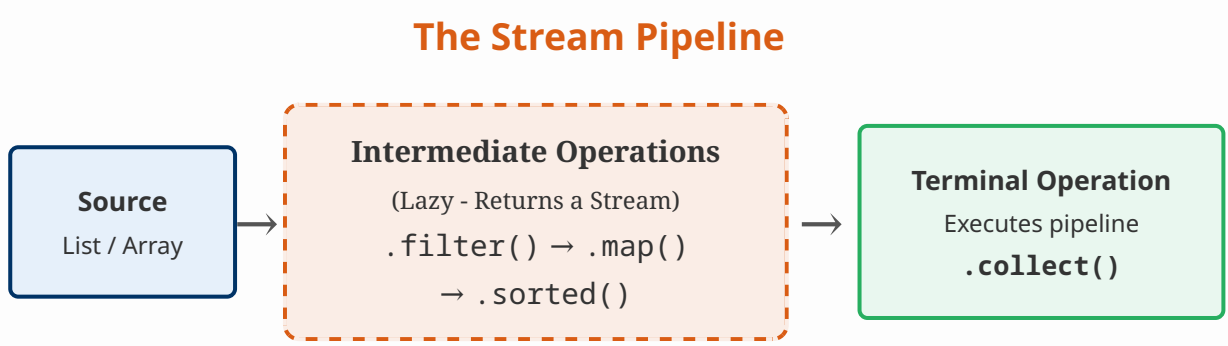
> **Common Mistakes:**
>
> - Trying to use a method reference when you need to manipulate the variable first. If you need to do `System.out.println(city + "!")`, you CANNOT use a method reference; you must use the full lambda.

> **Best Practices:**
>
> - Whenever your IDE suggests replacing a lambda with a method reference, do it. It is the modern standard.

---

7. The Stream API

A Stream in Java is a sequence of objects that supports various methods which can be pipelined to produce the desired result. Streams do NOT store data (like Collections do); they simply *process* data from a source.



Intermediate operations are LAZY. They don't actually do any work until a Terminal operation is called to pull the data through the pipe.

IT STU

Concept: Intermediate Operations: filter, map, sorted

Operations that transform or filter the stream and return a NEW stream, allowing them to be chained together.

> ***Real-Life Analogy:***
> *An assembly line in a factory. `filter()` removes the broken parts. `map()` paints the parts red. `sorted()` lines them up by size. The parts are just moving down the conveyor belt; they aren't boxed up yet.*

> **Simple Explanation:**
> `filter(Predicate)` keeps only items that return true. `map(Function)` transforms items into something else (e.g., turning Student objects into just Strings of their names). `sorted()` puts them in order.

Syntax / Structure:

```
list.stream()
    .filter(x -> x > 10)
    .map(x -> x * 2)
    .sorted()
```

Example Code:

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class StreamIntermediate {
    public static void main(String[] args) {
        List names = Arrays.asList("Alex", "Bob", "Charlie", "David", "Al", "Aaron");

        System.out.println("Original: " + names);

        // 1. Create stream from list
        List result = names.stream()
            // 2. Filter: Keep only names starting with 'A'
            .filter(name -> name.startsWith("A"))
            // 3. Map: Transform names to UPPERCASE
            .map(name -> name.toUpperCase())
```

```
            // 4. Sort: Alphabetical order
            .sorted()
            // 5. Terminal: Box them back into a List
            .collect(Collectors.toList());

        System.out.println("Processed: " + result);
    }
}
```

```
Expected Output:
Original: [Alex, Bob, Charlie, David, Al, Aaron]
Processed: [AARON, AL, ALEX]
```

Code Explanation:
We chained the methods beautifully. First, we threw out Bob, Charlie, and David. Then we uppercase the survivors. Then we sorted them. Finally, `.collect()` grabbed the results from the stream and built a brand new List.

Real-world Use:
Extracting a list of active user email addresses from a database of thousands of mixed User objects.

Common Mistakes:

- Trying to print or collect a stream *without* a terminal operation. If you just write `names.stream().filter(...)` without `.collect()`, literally nothing happens. Intermediate ops are lazy!

Best Practices:

- Format your stream pipelines vertically (one method per line) so they are highly readable.

Practice Task:
Create a List of Integers (1 to 5). Use a stream to `filter` out odd numbers, `map` them by multiplying by 10, and `.forEach(System.out::println)` to print the result.

Concept: Terminal Operations: collect, forEach, reduce

Operations that trigger the execution of the stream pipeline and produce a final, non-stream result.

> ***Real-Life Analogy:***
> *The end of the factory assembly line. `.collect()` puts the parts into a shipping box (a List). `.forEach()` inspects each part and throws it away (prints it). `.reduce()` melts all the parts down into one giant block of metal (a sum/total).*

> **Simple Explanation:**
> Once a terminal operation is called, the stream is consumed and closed. You cannot use that same stream again.

Syntax / Structure:

```
stream.collect(Collectors.toList());
stream.forEach(System.out::println);
stream.reduce(0, (a, b) -> a + b);
```

Example Code:

```
import java.util.Arrays;
import java.util.List;

public class StreamTerminal {
    public static void main(String[] args) {
        List prices = Arrays.asList(10, 20, 30, 40, 50);

        // Terminal 1: reduce() to calculate total sum
        // 0 is the starting value. (total, next) -> total + next
        int sum = prices.stream()
            .reduce(0, (total, current) -> total + current);

        System.out.println("Total Cart Price: $" + sum);

        // Terminal 2: count()
        long expensiveItems = prices.stream()
            .filter(p -> p > 25)
            .count();
```

```
        System.out.println("Items over $25: " + expensiveItems);
    }
}
```

```
Expected Output:
Total Cart Price: $150
Items over $25: 3
```

Code Explanation:
`.reduce()` takes a starting value (0) and a Lambda that defines how to combine two items. It adds 0+10, then 10+20, then 30+30, collapsing the entire stream into a single `int`. `.count()` simply tallies the remaining items.

> ***Real-world Use:***
> *Generating financial reports: `.reduce()` to sum up all transaction amounts for the day.*

> **Common Mistakes:**
> - Trying to reuse a stream. `Stream s = list.stream(); s.count(); s.collect();` will throw an `IllegalStateException` because the stream is already closed after `count()`.

> **Best Practices:**
> - Use `.collect(Collectors.toList())` when you need to save the data for later. Use `.forEach()` if you just want to immediately print or act on the data.

---

8. The Optional Class

IT STUDY HUB

Concept: Dealing with Nulls using Optional

A container object which may or may not contain a non-null value. It is used to avoid `NullPointerException`s.

> ***Real-Life Analogy:***
> *Instead of handing you a sandwich that might magically vanish and crash reality (Null), I hand you a sealed lunchbox (Optional). You must open the box and check if the sandwich is inside before trying to eat it.*

> **Simple Explanation:**
> In old Java, methods returned `null` if they couldn't find a result. If you tried to call a method on that `null`, your app crashed. `Optional` forces the programmer to acknowledge that the data might be missing, and provides safe methods like `.orElse()` to provide fallback data.

Syntax / Structure:

```
Optional opt = Optional.ofNullable(variable);
String result = opt.orElse("Fallback");
```

Example Code:

```
import java.util.Optional;

public class OptionalDemo {

    // Simulating a database lookup that might fail
    public static Optional findUser(int id) {
        if (id == 1) return Optional.of("Alice");
        if (id == 2) return Optional.of("Bob");
        return Optional.empty(); // No nulls returned!
    }

    public static void main(String[] args) {
        // Safe Retrieval
        Optional userOpt = findUser(3); // Does not exist

        // Using orElse to provide a default value if empty
        String userName = userOpt.orElse("Guest User");
```

```
        System.out.println("Welcome, " + userName);

        // Executing code ONLY if present
        Optional userOpt2 = findUser(1);
        userOpt2.ifPresent(u -> System.out.println("Found user: " + u));
    }
}
```

```
Expected Output:
Welcome, Guest User
Found user: Alice
```

Code Explanation:
`findUser(3)` returns `Optional.empty()`. When we call `.orElse("Guest User")`, it notices the box is empty and safely hands us the fallback string instead of crashing.

> ***Real-world Use:***
> *Database queries: `findById(userId)` returns `Optional`. If the user deleted their account, the system doesn't crash, it just renders an 'Account not found' page.*

> **Common Mistakes:**
>
> - Calling `.get()` on an Optional without checking `.isPresent()` first. If the box is empty, `.get()` will throw a `NoSuchElementException`, completely defeating the purpose of using Optional!

> **Best Practices:**
>
> - Never use Optional as a parameter in a method signature or as a class field. It was designed strictly to be used as a Return Type for methods that might fail to find a result.

---

9. Modern Date and Time API (java.time)

Before Java 8, dealing with dates using `java.util.Date` was notoriously difficult and prone to bugs. Java 8 introduced the `java.time` package, which is immutable, thread-safe, and highly intuitive.

Class	What it Stores	Example Format
LocalDate	Just a Date (No time, no timezone).	2026-08-15
LocalTime	Just a Time (No date, no timezone).	14:30:00.000
LocalDateTime	Both Date and Time.	2026-08-15T14:30:00

IT STUDY HUB

Concept: LocalDate and DateTimeFormatter

Fetching the current date, manipulating it, and formatting it for human readability.

> **Simple Explanation:**
> Because `LocalDate` is immutable, adding days to it returns a completely new `LocalDate` object. We use `DateTimeFormatter` to translate the strict ISO computer format into pretty strings like '15-Aug-2026'.

Syntax / Structure:

```
LocalDate today = LocalDate.now();
LocalDate future = today.plusDays(5);
```

Example Code:

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class DateTimeDemo {
    public static void main(String[] args) {
        // 1. Get current date
        LocalDate today = LocalDate.now();
        System.out.println("Raw ISO Date: " + today);

        // 2. Manipulate date (Immutable, so it returns a new object)
        LocalDate nextWeek = today.plusDays(7);
        LocalDate lastMonth = today.minusMonths(1);
        System.out.println("One week from now: " + nextWeek);

        // 3. Format Date
        // dd = day, MM = numeric month, MMM = short text month, yyyy = year
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MMM-yyyy");
        String formattedDate = today.format(formatter);
        System.out.println("Formatted Date: " + formattedDate);
    }
}
```

Expected Output:

```
Raw ISO Date: 2026-08-13
One week from now: 2026-08-20
Formatted Date: 13-Aug-2026
```

Code Explanation:
`LocalDate.now()` automatically fetches the system date. `.plusDays()` safely handles month rollovers (e.g., adding 5 days to Aug 28 safely becomes Sept 2 without manual math).

> ***Real-world Use:***
> *Calculating subscription expiration dates, scheduling system backups, or formatting timestamps on social media posts.*

> **Common Mistakes:**
>
> - Using a lowercase `m` instead of uppercase `M` in formatting. `mm` means Minutes, `MM` means Months! Formatting 'dd-mm-yyyy' will put random minute numbers in your date!

> **Best Practices:**
>
> - Always use `java.time.LocalDate` in new projects. Completely avoid the legacy `java.util.Date` and `java.util.Calendar` classes.

---

10. Java 14+ Features: Records and Pattern Matching

IT STUDY HUB

Concept: Records (Java 14)

A compact syntax for declaring immutable data carrier classes.

> ***Real-Life Analogy:***
> *If a standard Class is a complex customized contract drafted by a lawyer, a Record is a simple fill-in-the-blank post-it note. It cuts out the legal jargon.*

> **Simple Explanation:**
> Often, we create classes just to hold data (like a `Student` with ID and Name), forcing us to write getters, constructors, `toString()`, and `equals()` methods—dozens of lines of boring boilerplate. The `record` keyword auto-generates all of this for you instantly. Records are strictly immutable.

Syntax / Structure:

```
public record RecordName(Type field1, Type field2) { }
```

Example Code:

```
// This ONE line replaces ~30 lines of standard Class code!
record StudentRecord(int id, String name, double gpa) { }

public class RecordDemo {
    public static void main(String[] args) {

        // 1. Constructor is auto-generated
        StudentRecord s1 = new StudentRecord(101, "Alice", 3.8);
        StudentRecord s2 = new StudentRecord(101, "Alice", 3.8);

        // 2. Getters are auto-generated (without the 'get' prefix)
        System.out.println("Student Name: " + s1.name());
        System.out.println("Student GPA: " + s1.gpa());

        // 3. toString() is auto-generated beautifully
        System.out.println("Object Dump: " + s1);

        // 4. equals() is auto-generated to compare data, not memory!
        System.out.println("Are s1 and s2 identical? " + s1.equals(s2));
```

```
    }
}
```

```
Expected Output:
Student Name: Alice
Student GPA: 3.8
Object Dump: StudentRecord[id=101, name=Alice, gpa=3.8]
Are s1 and s2 identical? true
```

Code Explanation:
We didn't write a constructor, getters, `toString`, or `equals` inside `StudentRecord`. Java generated them all at compile-time. Note that getter methods in Records do not use the word 'get' (it's `s1.name()`, not `s1.getName()`).

> ***Real-world Use:***
> *Used heavily in fetching Data Transfer Objects (DTOs) from databases or JSON APIs where data just needs to be read and transported, not modified.*

> **Common Mistakes:**
>
> - Trying to modify a Record's data after creation (e.g., `s1.name = "Bob";`). Records are 100% immutable and have no Setter methods.

> **Best Practices:**
>
> - Use standard Classes if the object's state needs to change over time (like a BankAccount). Use Records if the object is just a snapshot of data (like a TransactionReceipt).

---

11. Project: Java Data-Processing Application

Let's synthesize Streams, Lambdas, Records, and Optional. We will process a database of Employees. We will filter out low performers, give raises to the top performers, sort them, and safely search for a specific employee.

```
import java.util.Arrays;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

// 1. Using a Record for clean, immutable data modeling
record Employee(int id, String name, String department, double salary) {}

public class EnterpriseDataProcessor {
    public static void main(String[] args) {

        // 2. Mock Database
        List staff = Arrays.asList(
            new Employee(1, "Alice", "IT", 75000),
            new Employee(2, "Bob", "HR", 50000),
            new Employee(3, "Charlie", "IT", 90000),
            new Employee(4, "Diana", "Finance", 60000),
            new Employee(5, "Eve", "IT", 45000)
        );

        System.out.println("=== ENTERPRISE DATA PROCESSOR ===");

        // 3. STREAM PIPELINE: Filter, Map, Sort, Collect
        System.out.println("\n--- IT Department Highest Earners (After 10% Raise) ---");

        List topITStaff = staff.stream()
            // Predicate: Keep only IT employees
            .filter(emp -> emp.department().equals("IT"))
            // Predicate: Keep only those making >= 60k
            .filter(emp -> emp.salary() >= 60000)
            // Function: Transform Employee object into a formatted String with a 10% ra
            .map(emp -> emp.name() + " : $" + (emp.salary() * 1.10))
            // Sort strings alphabetically
            .sorted()
```

```
            // Terminal: Collect into List
            .collect(Collectors.toList());

        // Consumer: Print the results using Method Reference
        topITStaff.forEach(System.out::println);

        // 4. STREAM PIPELINE: Reduce (Total Payroll)
        System.out.println("\n--- Company Metrics ---");
        double totalPayroll = staff.stream()
            // Extract just the salaries
            .map(emp -> emp.salary())
            // Sum them all up (Start at 0.0)
            .reduce(0.0, (total, current) -> total + current);

        System.out.println("Total Monthly Payroll: $" + totalPayroll);

        // 5. OPTIONAL: Safe Searching
        System.out.println("\n--- Employee Search ---");
        int searchId = 3;

        Optional foundEmp = staff.stream()
            .filter(emp -> emp.id() == searchId)
            .findFirst(); // Terminal operation returning Optional

        // Safe extraction
        foundEmp.ifPresentOrElse(
            emp -> System.out.println("Found: " + emp.name() + " in " + emp.department()),
            () -> System.out.println("Error: Employee ID " + searchId + " not found.")
        );
    }
}
```

```
=== ENTERPRISE DATA PROCESSOR ===

--- IT Department Highest Earners (After 10% Raise) ---
Alice : $82500.0
Charlie : $99000.0

--- Company Metrics ---
Total Monthly Payroll: $320000.0
```

```
--- Employee Search ---
Found: Charlie in IT
```

Step-by-Step Execution:

1. **Records:** We defined `Employee` in one line. We use auto-generated getters like `emp.department()` inside the lambdas.
2. **First Pipeline:** We chained two `.filter()` methods to narrow the list down to Alice and Charlie. We used `.map()` to calculate their raises and transform the data into pure text Strings. We collected it, and printed using `System.out::println`.
3. **Reduce:** We mapped the Employees into just a stream of `Double`s (salaries). We used `.reduce()` to iteratively add them all to a running total.
4. **Optional:** `findFirst()` looks for the first match and wraps it in an `Optional`. `ifPresentOrElse` runs the first lambda if the data exists, or the second lambda (a Supplier) if it is missing, preventing any NullPointerExceptions!

IT STUDY HUB

12. Module Assessment and Interview Prep

Test your mastery of Functional Programming, Streams, and Modern Java to prepare for technical interviews.

A. Practice Questions (Test Your Logic)

1. Write a Lambda expression that takes two integers and returns their product. Assign it to a custom Functional Interface.
2. Use the Stream API on a List of integers (1 to 10) to `filter` only even numbers, and `forEach` to print them.
3. Convert this lambda to a Method Reference: `name -> name.toUpperCase()` (Assume the class is String).
4. What is the difference between an intermediate operation and a terminal operation in Streams?
5. Write a Stream pipeline that takes a List of Strings, uses `.map()` to get their lengths, and uses `.reduce()` to find the total sum of all lengths.
6. Why does calling `.filter()` without `.collect()` or `.forEach()` result in nothing happening?
7. Write code using `Optional.ofNullable(variable)`. Use `.orElse("Unknown")` to safely assign it to a String.
8. Write a Java program to get the current date using `LocalDate` and add 2 weeks to it.
9. Declare a Java `record` named `Book` with fields for title and price. Instantiate it and print the title.
10. Can you change the value of a field inside a `record` object after it has been created? Why or why not?
11. Explain the difference between a `Predicate` and a `Consumer`.
12. What terminal Stream operation would you use to count the number of items that passed a filter?
13. Use `DateTimeFormatter` to format the current date as "yyyy/MM/dd".
14. Identify the error: `Optional<String> opt = Optional.empty(); System.out.println(opt.get());`

IT STUDY HUB

15. If a Functional Interface has one abstract method and one `default` method (with a body), is it still a valid Functional Interface?

B. University Viva Questions & Answers

Q: What is Functional Programming?

Ans: A programming paradigm where programs are constructed by applying and composing functions, treating them as first-class citizens that can be assigned to variables and passed as arguments.

Q: What is a Lambda Expression?

Ans: A concise, anonymous function (without a name, return type declaration, or access modifier) that implements the single abstract method of a Functional Interface.

Q: What is a Functional Interface?

Ans: An interface that contains exactly one abstract method. It can contain any number of default or static methods. It is the target type for lambda expressions.

Q: What does the @FunctionalInterface annotation do?

Ans: It informs the compiler to enforce the rule that the interface must have exactly one abstract method, causing a compilation error if a second one is added.

Q: Explain the Predicate interface.

Ans: It is a built-in functional interface that takes one argument and returns a boolean. It is primarily used for filtering data.

Q: Explain the Consumer interface.

Ans: It takes one argument and returns no result (void). It is used to perform side-effects, like printing data to the console.

Q: Explain the Function interface.

Ans: It takes one argument of type T and transforms it into a result of type R. It is heavily used in the Stream .map() operation.

Q: What is a Method Reference?

Ans: A shorthand syntax (::) for a lambda expression that simply calls an existing method. For example, System.out::println instead of x -> System.out.println(x).

IT STUDY HUB

Q: What is the Java Stream API?

Ans: A sequence of elements supporting sequential and parallel aggregate operations. It allows declarative data processing (filtering, mapping, reducing) on collections.

Q: What is the difference between Intermediate and Terminal operations?

Ans: Intermediate operations (filter, map) transform a stream and return a new stream; they are lazy and don't execute immediately. Terminal operations (collect, count, forEach) trigger the actual execution of the pipeline and return a non-stream result.

Q: What does it mean that Streams are 'lazy'?

Ans: It means computation on the source data is only performed when a terminal operation is initiated. If no terminal operation is present, the intermediate operations are completely ignored.

Q: Can a Stream be reused?

Ans: No. Once a terminal operation has been called on a Stream, it is considered consumed and closed. Attempting to use it again throws an IllegalStateException.

Q: What is the purpose of the Optional class?

Ans: It is a container object used to represent the presence or absence of a value, providing a safe and elegant alternative to returning 'null' and preventing NullPointerExceptions.

Q: Why is java.time (LocalDate) better than the old java.util.Date?

Ans: The new java.time classes are immutable and thread-safe, whereas the old Date class was mutable, thread-unsafe, and had poor, confusing indexing (like months starting at 0).

Q: What is a Java Record?

Ans: Introduced in Java 14, a Record is a special kind of class designed to hold immutable data. It automatically generates boilerplate code like constructors, getters, equals(), hashCode(), and toString().

Q: Are Record fields mutable?

Ans: No. All fields in a Record are implicitly 'final' and cannot be modified after instantiation.

Q: What is a Sealed Class? (Briefly)

Ans: A modern Java feature that restricts which other classes may extend it. You declare a class 'sealed' and use the 'permits' keyword to explicitly list the allowed subclasses.

IT STUDY HUB

Q: What does the .map() operation do in a Stream?

Ans: It applies a given Function to each element in the stream, transforming the elements into a new form (e.g., converting a stream of Objects into a stream of their integer IDs).

Q: What does the .reduce() operation do?

Ans: It performs a reduction on the elements of the stream, repeatedly combining them using a provided operation (like addition) to compress the stream into a single summary value.

Q: What happens if you call .get() on an empty Optional?

Ans: It throws a NoSuchElementException. You should always use .isPresent() or .orElse() instead of calling .get() blindly.

C. Intermediate Interview Questions

Interview Q: How does Java implement Lambda Expressions internally? Does it create an Anonymous Inner Class?

Ans: No, it doesn't create traditional anonymous inner classes (.class files). Java 7 introduced 'invokedynamic'. The JVM dynamically generates the implementation of the functional interface at runtime using MethodHandles, making lambdas significantly faster and more memory-efficient than inner classes.

Interview Q: Can a lambda expression access variables outside its scope?

Ans: Yes, it can access instance variables, static variables, and local variables. However, local variables accessed by a lambda must be 'effectively final'—meaning their value cannot be changed after initialization, otherwise the compiler will throw an error.

Interview Q: What is the difference between .map() and .flatMap() in Streams?

Ans: .map() transforms one object into another object (1-to-1 ratio). .flatMap() transforms one object into a Stream of multiple objects, and then 'flattens' all those nested streams into one single continuous stream (1-to-Many ratio). It is used to unravel Lists inside of Lists.

Interview Q: Why is mutating state inside a .forEach() or .map() dangerous?

Ans: Streams are designed for functional programming, which emphasizes pure, side-effect-free functions. Mutating external variables inside a stream breaks thread-safety. If you switch the stream to .parallelStream(), concurrent mutations will cause catastrophic data corruption and race conditions.

IT STUDY HUB

Interview Q: Explain the difference between findFirst() and findAny().

Ans: findFirst() guarantees returning the very first element in the ordered stream. findAny() returns whatever element it happens to find first. In a sequential stream, they behave identically. In a parallel stream, findAny() is much faster because it grabs the first result from any thread, ignoring strict ordering.

Interview Q: What is 'Short-circuiting' in Streams?

Ans: Stream operations like .limit(n), .findFirst(), or .anyMatch() do not need to process the entire dataset. Once the condition is met (e.g., finding one matching item), the stream pipeline instantly aborts processing the remaining elements, saving massive processing time.

Interview Q: How does Optional.orElseGet() differ from Optional.orElse()?

Ans: .orElse(foo()) will execute the foo() method to evaluate the fallback value regardless of whether the Optional is empty or full (wasteful). .orElseGet(() -> foo()) uses a Supplier lambda, meaning it is LAZY—it only executes foo() if the Optional is actually empty, saving performance.

Interview Q: Why can't Records extend other classes?

Ans: Records implicitly extend the 'java.lang.Record' class. Since Java does not support multiple class inheritance, a Record cannot extend any other class. However, Records CAN implement interfaces.

Interview Q: What is the advantage of using Parallel Streams?

Ans: By calling .parallelStream() instead of .stream(), Java automatically splits the data processing across multiple CPU cores using the ForkJoin pool. This can drastically reduce processing time for massive datasets, provided the operations are stateless and non-interfering.

Interview Q: When would a Parallel Stream actually be SLOWER than a sequential Stream?

Ans: Parallel streams have overhead: splitting the data, managing threads, and merging the results. For small collections, or operations that require strict ordering, the thread-management overhead takes longer than just processing the data sequentially on a single thread.

IT STUDY HUB

14. Module Summary and Key Terms

Key Terms Glossary

- **Lambda Expression:** An anonymous function syntax `()->{}` used to implement Functional Interfaces concisely.
- **Functional Interface:** An interface strictly containing only one abstract method.
- **Stream:** A pipeline tool that processes sequences of elements using functional operations.
- **Lazy Evaluation:** Intermediate stream operations do not process data until a terminal operation demands it.
- **Optional:** A container object that forces safe handling of potentially `null` values.
- **Record:** A compact Java class designed specifically to hold immutable data with zero boilerplate.

Module Summary

In this module, you stepped into the modern era of Java development. You replaced clunky, verbose code with elegant **Lambda Expressions**, understanding the underlying mechanics of `Predicate`, `Consumer`, and `Function` interfaces.

You unlocked the immense power of the **Stream API**, learning to chain `filter`, `map`, and `sorted` intermediate operations before executing them with `collect` or `reduce`. This allows you to process massive datasets in a single, readable statement. You eliminated the dread of NullPointerExceptions using **Optional**, modernized your date logic with the **java.time** API, and drastically reduced class boilerplate using **Records**.

The Enterprise Data Processor project synthesized these concepts, proving you can write clean, functional, and highly optimized modern Java code.

WHAT COMES NEXT?

MODULE 9 — MULTITHREADING & CONCURRENCY

Everything we have written so far has executed sequentially on a single thread—the CPU does one task at a time. Real-world software, like web servers or video games, must do thousands of things simultaneously. In the next volume, you will master **Multithreading**. You will learn to spawn multiple threads, execute code concurrently, and tackle complex synchronization issues to prevent race conditions and deadlocks.

IT STUDY HUB SERIES